IT342-G01 - SYSTEMS INTEGRATION AND ARCHITECTURE 1

Library Resource Booking and Management System

Core Feature Development and Implementation Report

Prepared by:	Jian Brenz C. Becera
Institution:	Cebu Institute of Technology University
Course:	IT342 - Systems Integration and Architecture 1 (G01)
GitHub Repository:	https://github.com/JBecera/Becera_LMS_backend
Report date:	July 17, 2026

This report documents the core-feature development cycle that closed the remaining gaps between the approved Software Requirements Specification (SRS) and the implemented system: the backend REST API (Spring Boot), the web client (React), and the mobile client (Kotlin/Jetpack Compose). It covers newly implemented functional requirements, how the three components integrate and exchange data, screenshots of the working system across all three roles (Member, Librarian, Admin), the results of three complete end-to-end workflow tests, integration issues found and fixed during testing, and the full commit history for this development cycle.

1. Development Progress Summary

At the start of this development cycle, the project had a partially-built web frontend (pages and service calls already written for reservations, transactions, and fines) with no corresponding backend implementation, no real authentication/authorization enforcement, and a mobile app that was little more than a login screen. This cycle closed those gaps end to end:

- Backend: implemented the Transaction (check-in/check-out), Reservation, and Fine domains that the frontend was already calling, added JWT-based authentication and role-based authorization (previously every endpoint was open to any unauthenticated caller), and removed a dead/duplicate code scaffold left over from an earlier prototype.
- Frontend: wired the JWT token through every API call, corrected the catalog reservation flow to match the SRS business rule (reserve only once a title is fully checked out), and added the missing self-service account page.
- Mobile: added session persistence and a full set of member-facing screens (catalog, my borrowing, reservations, fines, account) with a bottom-navigation shell, bringing the mobile app to functional parity with the member side of the web app.
- Testing: added three automated, full-stack end-to-end workflow tests plus unit tests for every new service; 21 backend tests pass. Browser-driven testing of the real running application (not just API calls) additionally surfaced and fixed two integration bugs invisible to API-level testing (Section 6).

All twelve commits in this cycle are organized by feature/integration task with descriptive messages (Section 8).

2. Description of Newly Implemented Features

FR-004: Admin Access / Authorization

Real JWT authentication replaces the previous open configuration. Every endpoint is now enforced server-side by role: public catalog browsing and self-registration stay open, member-owned resources (own profile, own loans, own reservations, own fines) require the caller to either be the owner or staff, and catalog/member/checkout/approval/settlement actions require LIBRARIAN or ADMIN. Self-registration can no longer request role=ADMIN - the server always forces MEMBER unless the caller is an authenticated ADMIN.

FR-003: Manage/View Account

A new self-service Account page (web and mobile) lets a member view their generated Student ID and update phone number, address, email, and password. Password fields are never returned by the API and are only changed when a new value is actually submitted.

FR-005: Register Students

Librarians register students from the Students page; each new member gets a generated, unique Student ID (STU-000123 format). Only an ADMIN caller may create LIBRARIAN or ADMIN accounts; a LIBRARIAN or anonymous caller is always forced to MEMBER.

FR-007: Check In / Check Out

Librarians check a catalog title out to a member (Transactions page), enforcing: no unpaid fines, no existing overdue items, a maximum of 3 active loans, and copy availability. Checking a title back in restores availability and, if returned past its due date, automatically issues a Fine.

FR-008: Reserve Resource

Members can only reserve a title once every copy is checked out (the literal SRS rule - the previous prototype had this backwards). Duplicate active reservations for the same member/title are rejected. Librarians approve or reject from a pending queue.

Fines & Business Rules

Overdue fines are generated automatically at check-in. Unpaid fines and overdue items block new checkouts and reservations until settled, matching the SRS business rules.

3. Feature Integration

The three components share one contract: the Spring Boot REST API. All data - catalog, members, transactions, reservations, fines - lives in a single PostgreSQL database (Supabase), and every client (web, mobile, and any future client) reaches it exclusively through the same JSON/HTTP endpoints secured by the same JWT.

Cross-feature backend integration

The Transaction, Reservation, and Fine services are not independent - they read and write each other's data to enforce the SRS business rules as one system rather than three separate features: checkout consults FineRepository (block on unpaid fines) and TransactionRepository (block on overdue items) before creating a loan; check-in writes a Fine when a return is late; reservation creation consults the same fine/overdue checks plus ReservationRepository for duplicate prevention; and a successful checkout automatically marks any matching pending/approved reservation as completed.

Backend <-> Web integration

The React app's service layer (api.js) attaches the JWT as a Bearer token to every request via an axios interceptor. Response DTOs on the backend (TransactionResponse, ReservationResponse, FineResponse) flatten the Book/Member relations into the exact field shape (resourceId/resourceTitle/memberName/etc.) the frontend pages already expected, so the UI and API share one data contract with no client-side field mapping.

Backend <-> Mobile integration

The Kotlin app's Retrofit interfaces (BookApi, TransactionApi, ReservationApi, FineApi, MemberApi) target the same endpoints and JSON shapes as the web client. SessionManager persists the JWT via DataStore and an OkHttp interceptor attaches it to every Retrofit call, so a member's session survives an app restart and every mobile screen reads/writes the same live data a librarian sees on the web app in real time.

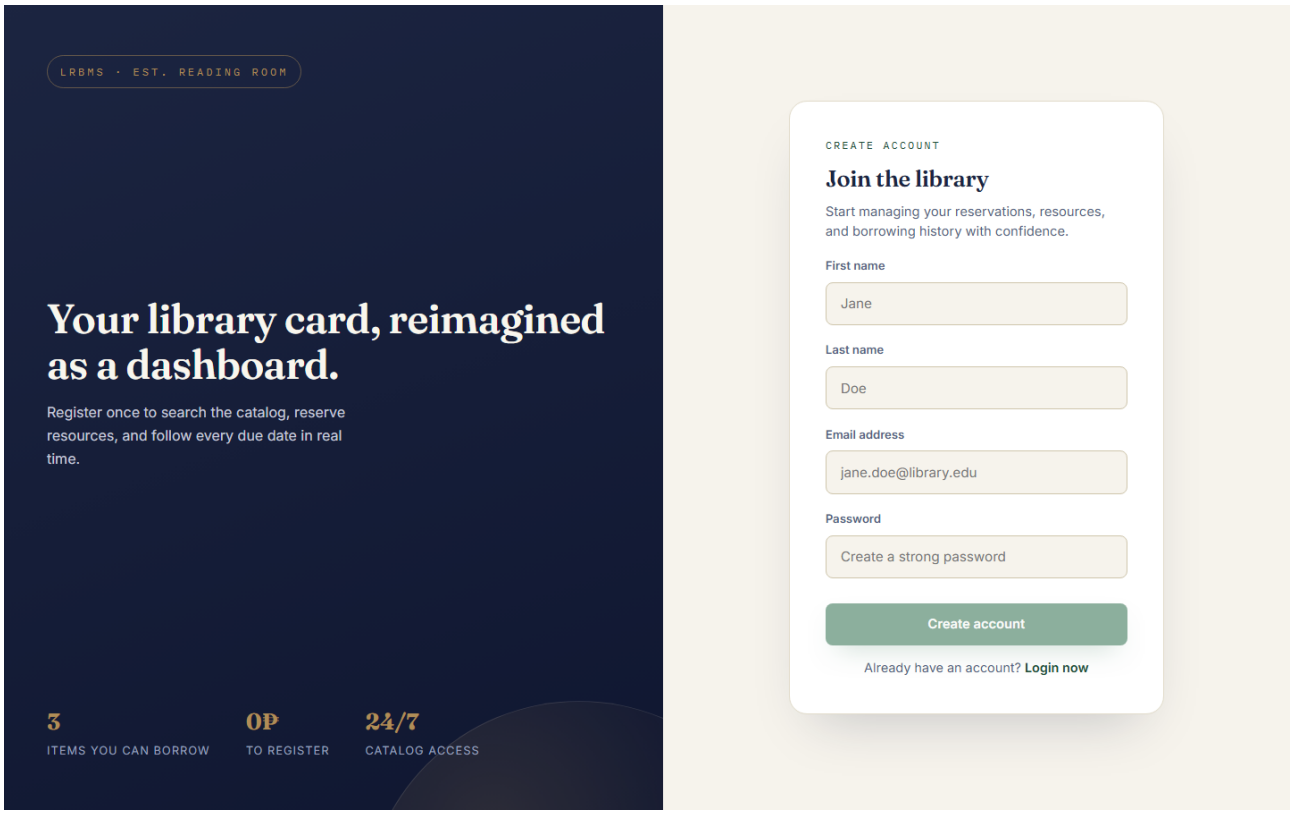
Consistency across clients

Because both clients call the same backend business logic, a title a librarian checks out on the web app immediately shows as reduced availability in the mobile Catalog screen, and a fine issued by the backend on an overdue check-in appears in both the web Fines page and the mobile Fines screen for that member - there is no duplicated or diverging business logic on either client.

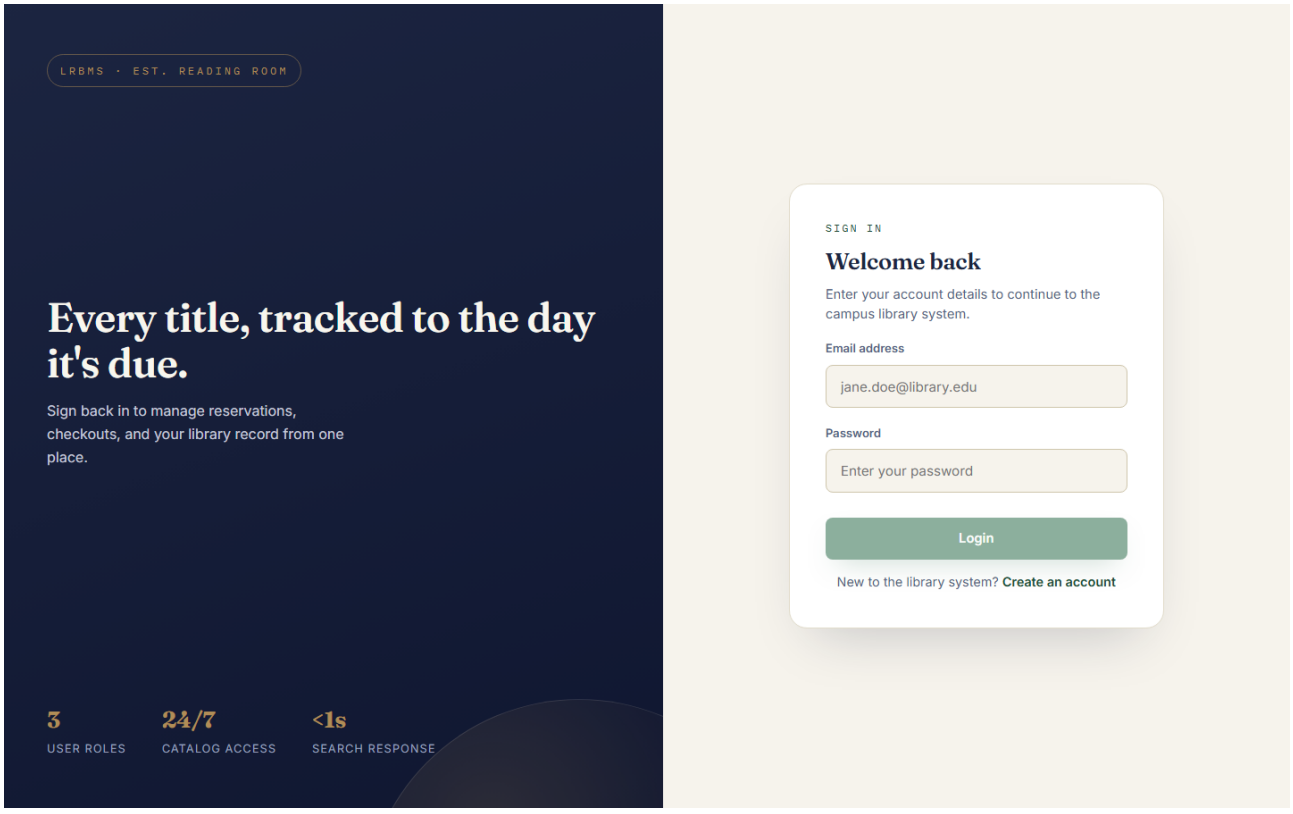
4. Screenshots of Working Functionality

Captured from the running application (Spring Boot backend + Vite dev server) via a headless-Chromium walk-through of every implemented screen, logged in as each of the three roles in turn. Demo data (a sample member "Maria Santos" with an active loan, borrowing history, a settled fine, and a pending reservation; a second member "Jake Reyes" with an unpaid fine) was seeded beforehand so every screen shows real, non-empty state.

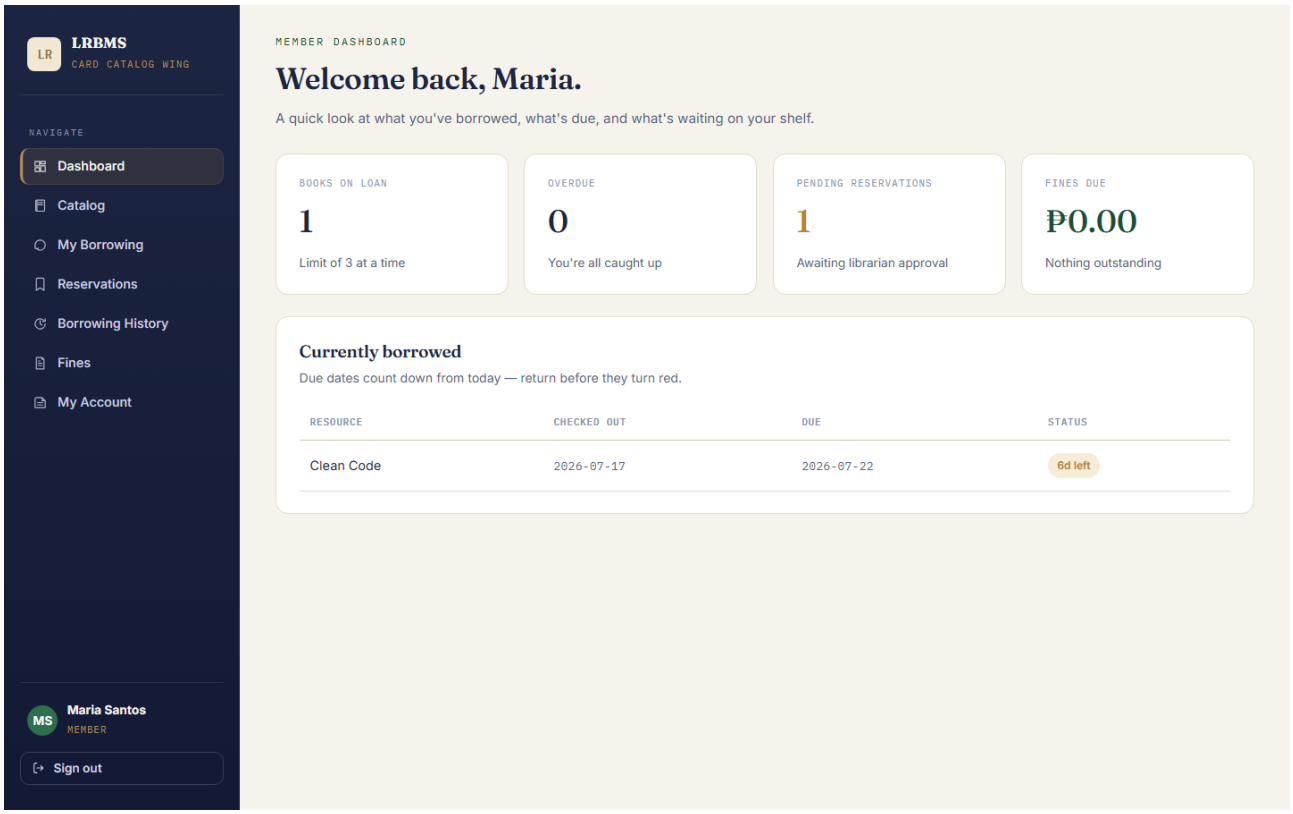
4.1 Public registration page (FR-005 self-service signup)



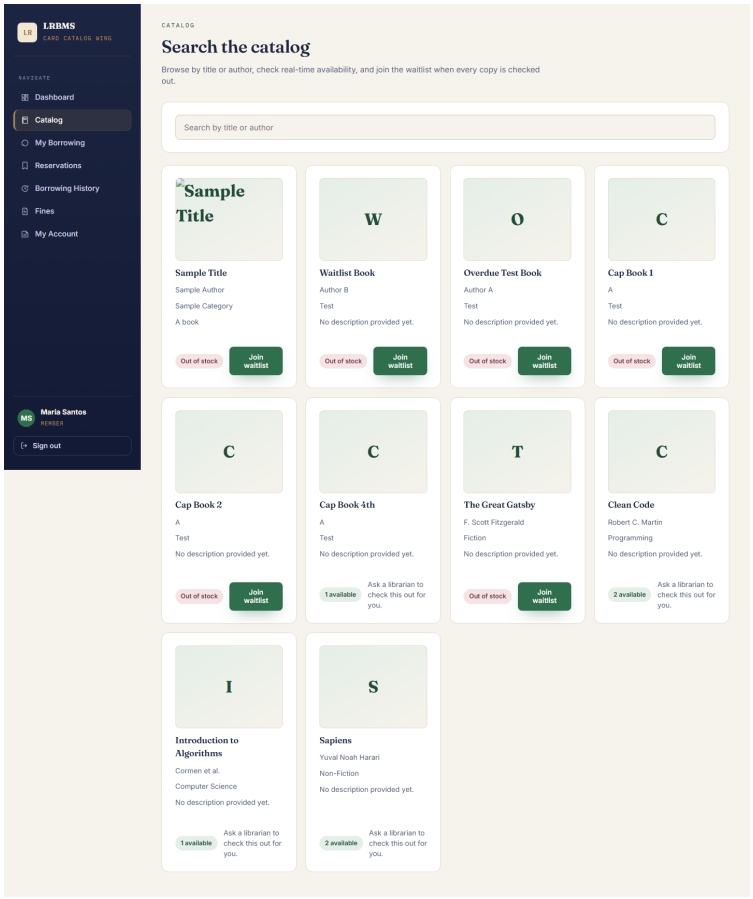
4.2 Login page (FR-001)



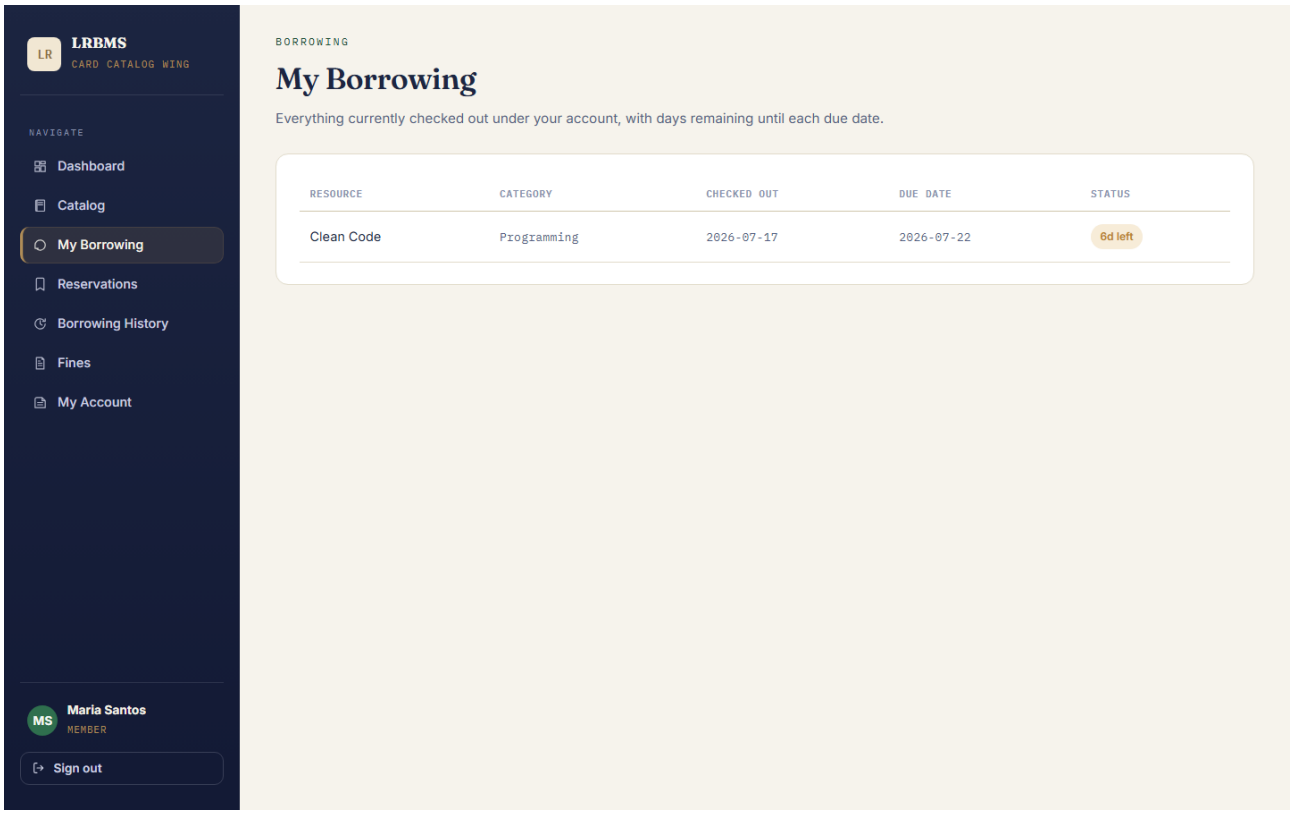
4.3 Member Dashboard - active loans, pending reservations, fines due



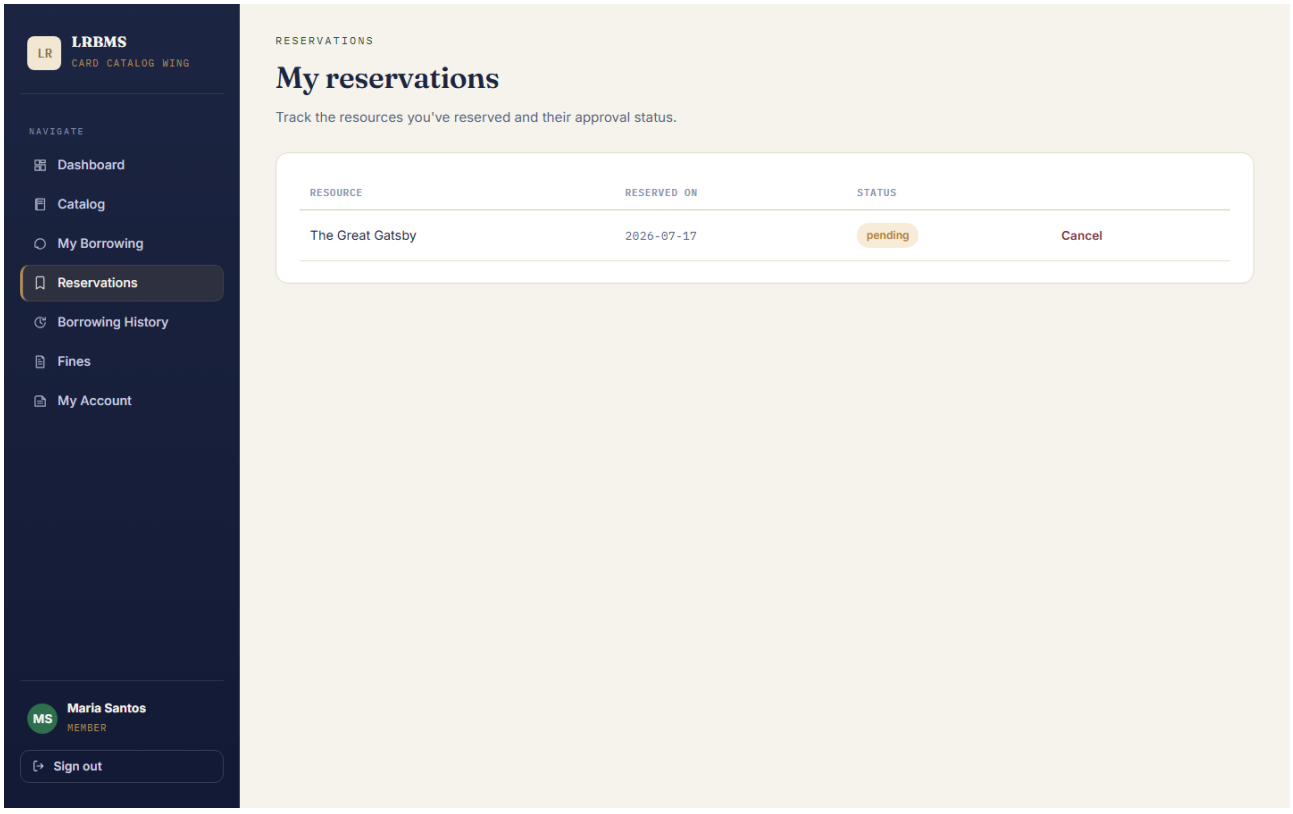
4.4 Catalog search (FR-002) - waitlist join once a title is fully checked out (FR-008)



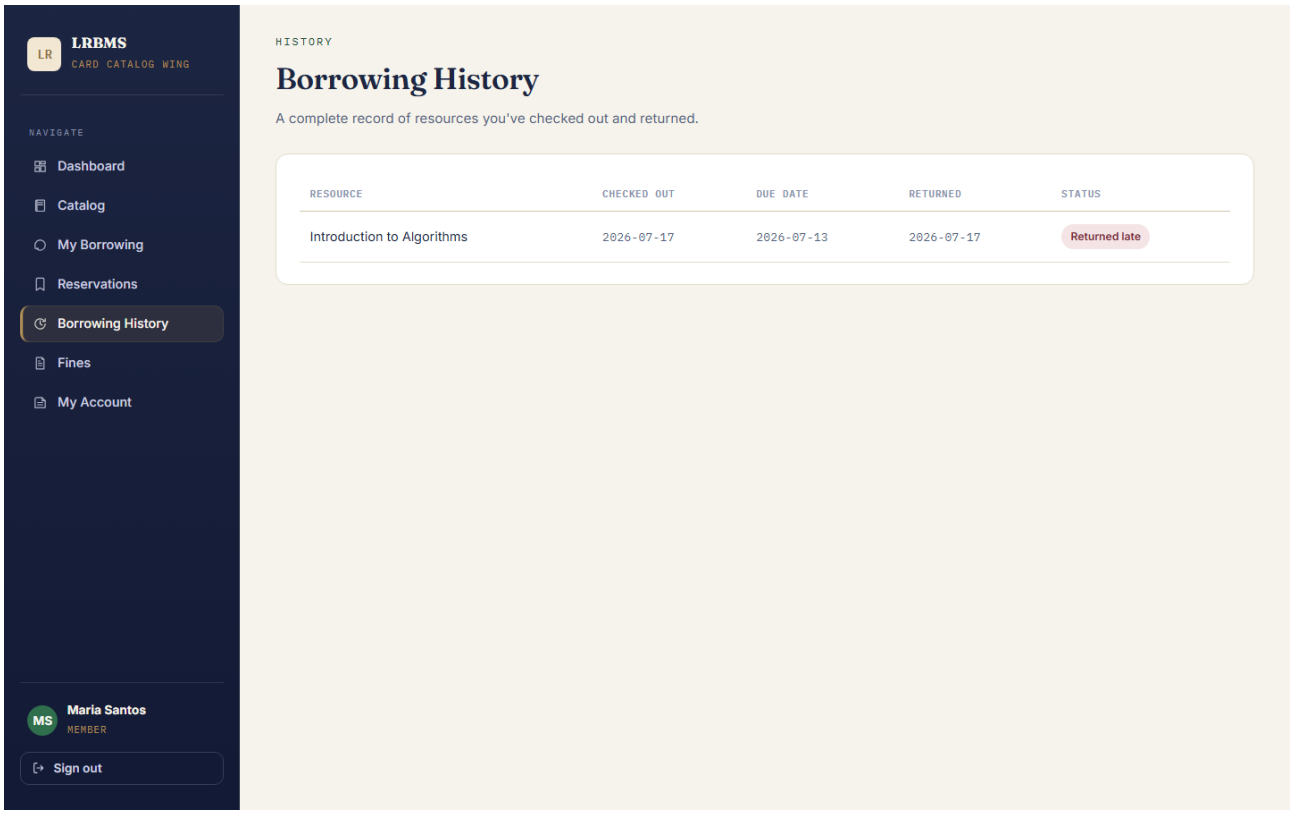
4.5 My Borrowing - active loan with live due-date countdown (FR-007)



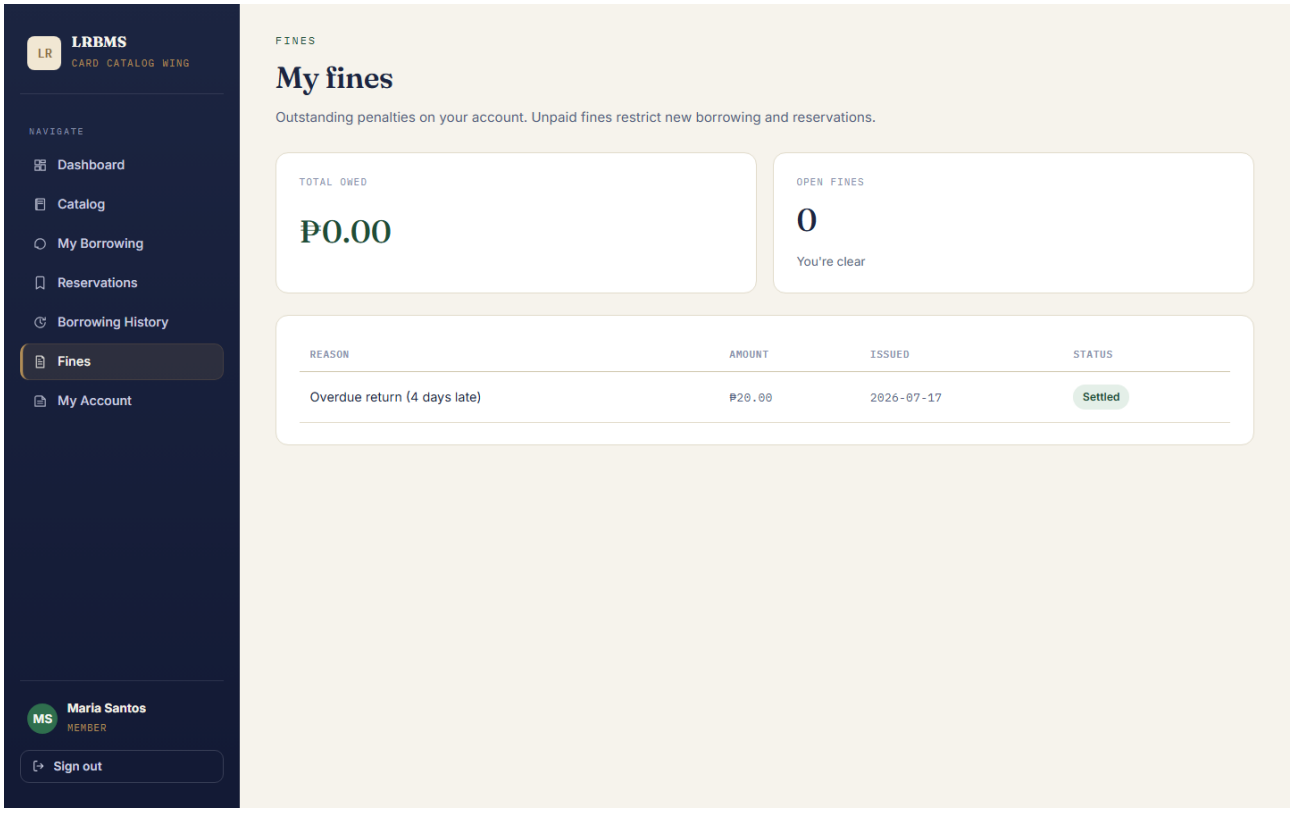
4.6 My Reservations - pending waitlist entry (FR-008)



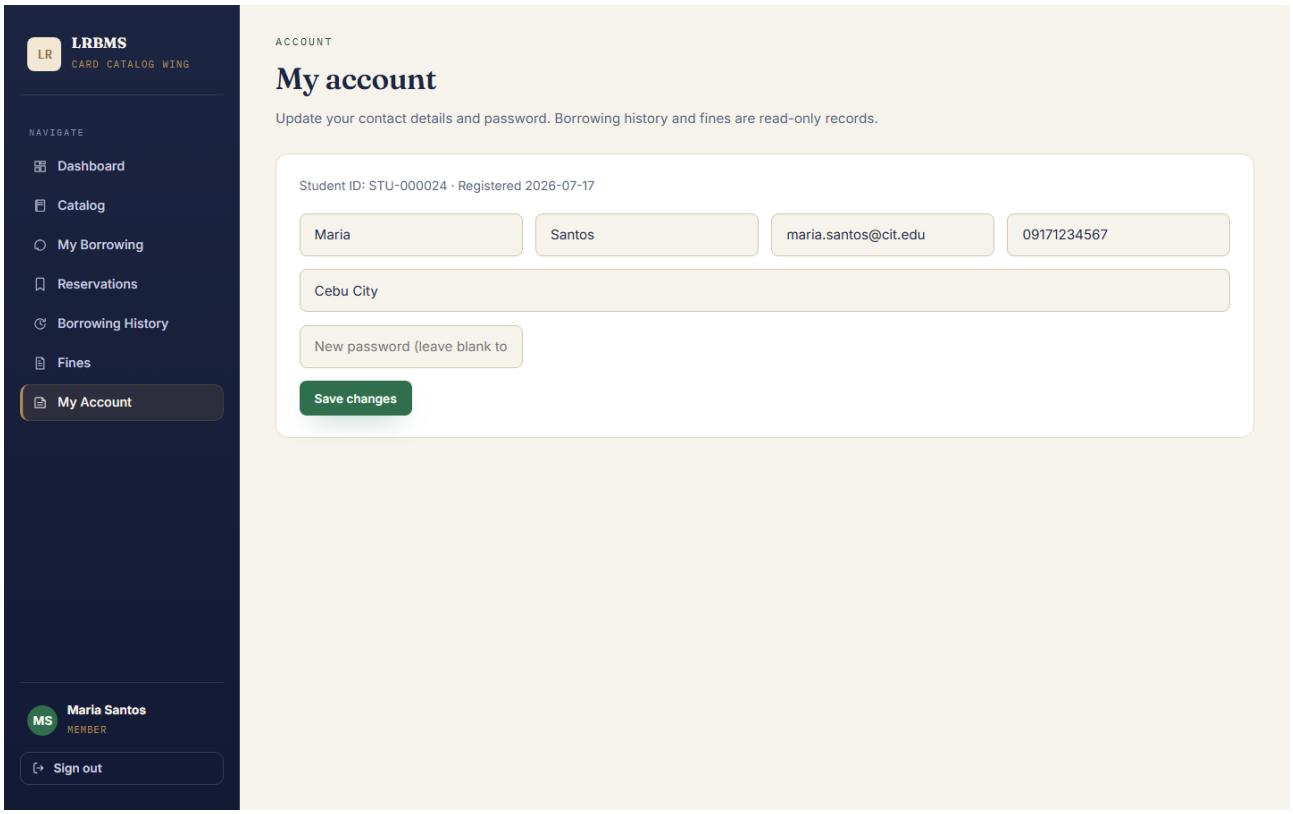
4.7 Borrowing History - past returns



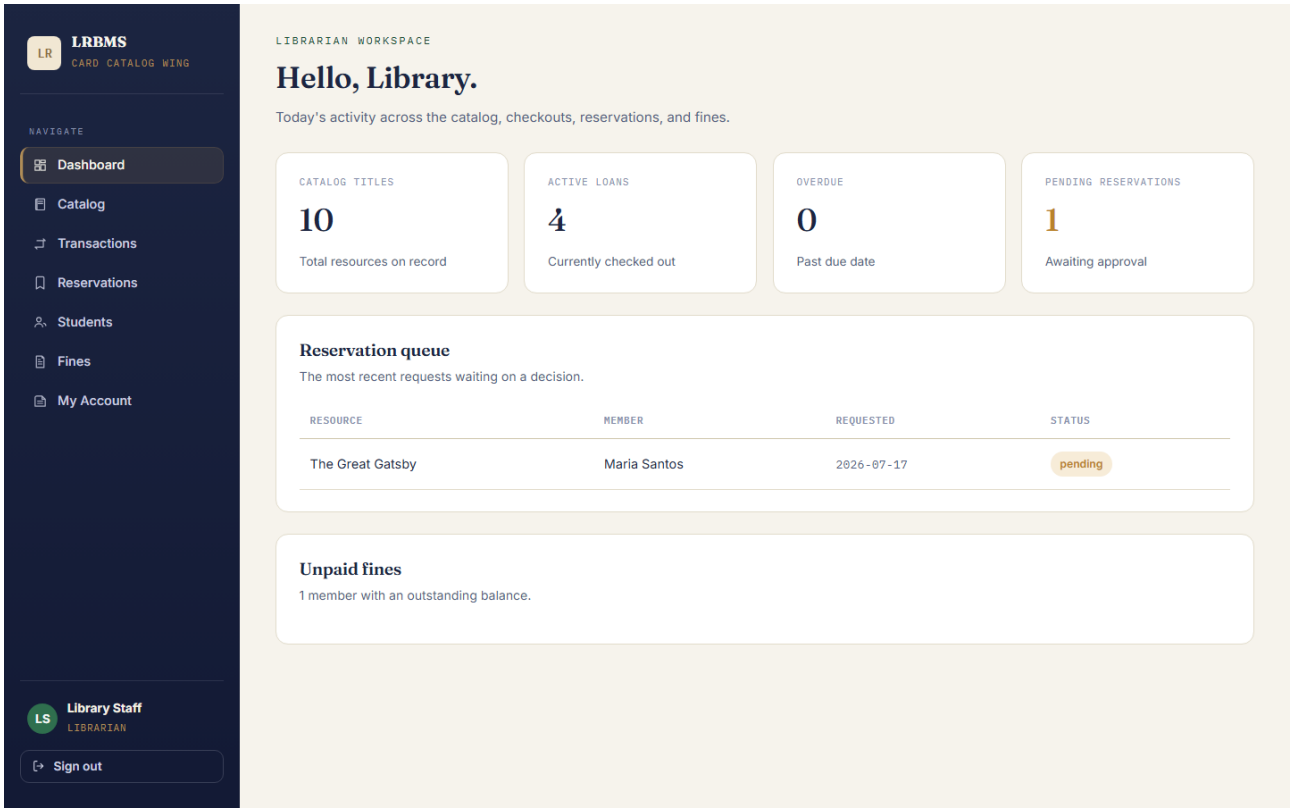
4.8 My Fines - settled fine on record



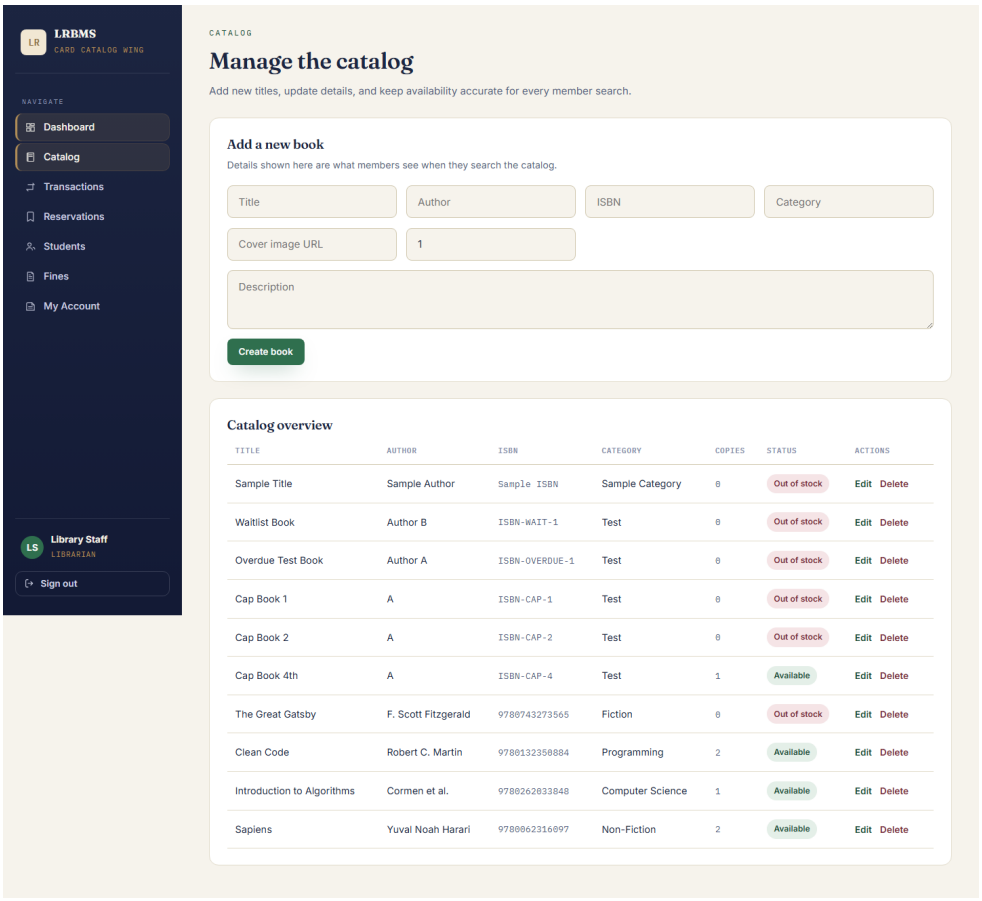
4.9 My Account - self-service profile update (FR-003)



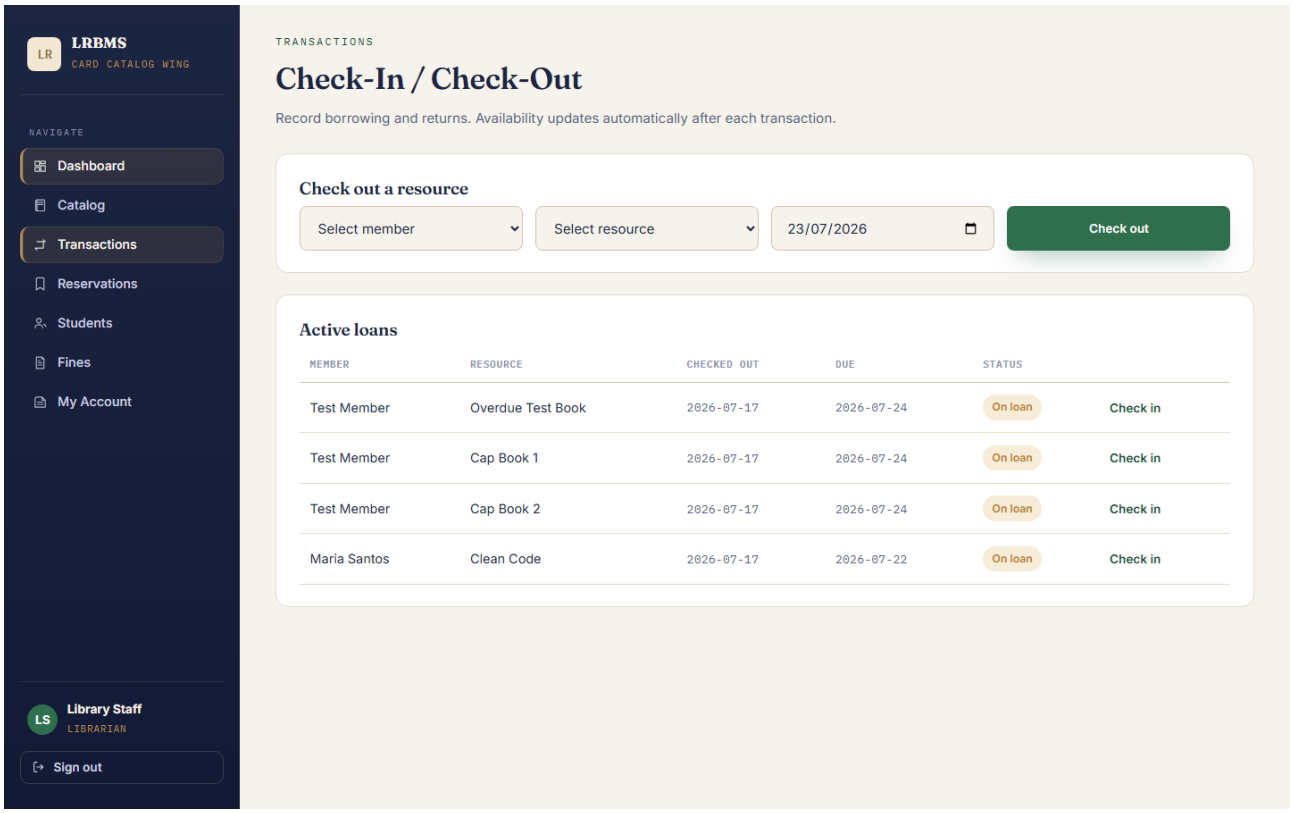
4.10 Librarian Dashboard - system-wide activity overview



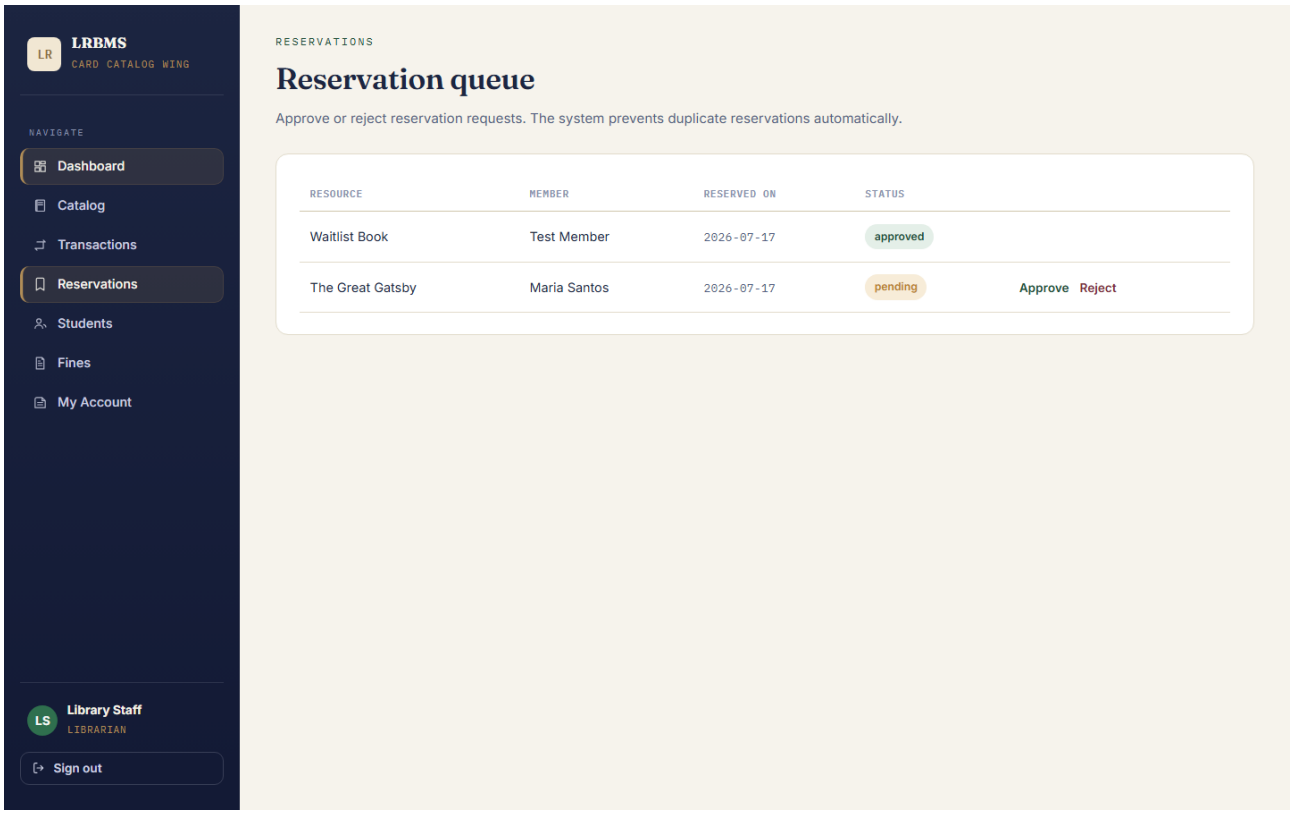
4.11 Manage Catalog - librarian CRUD over titles (FR-006)



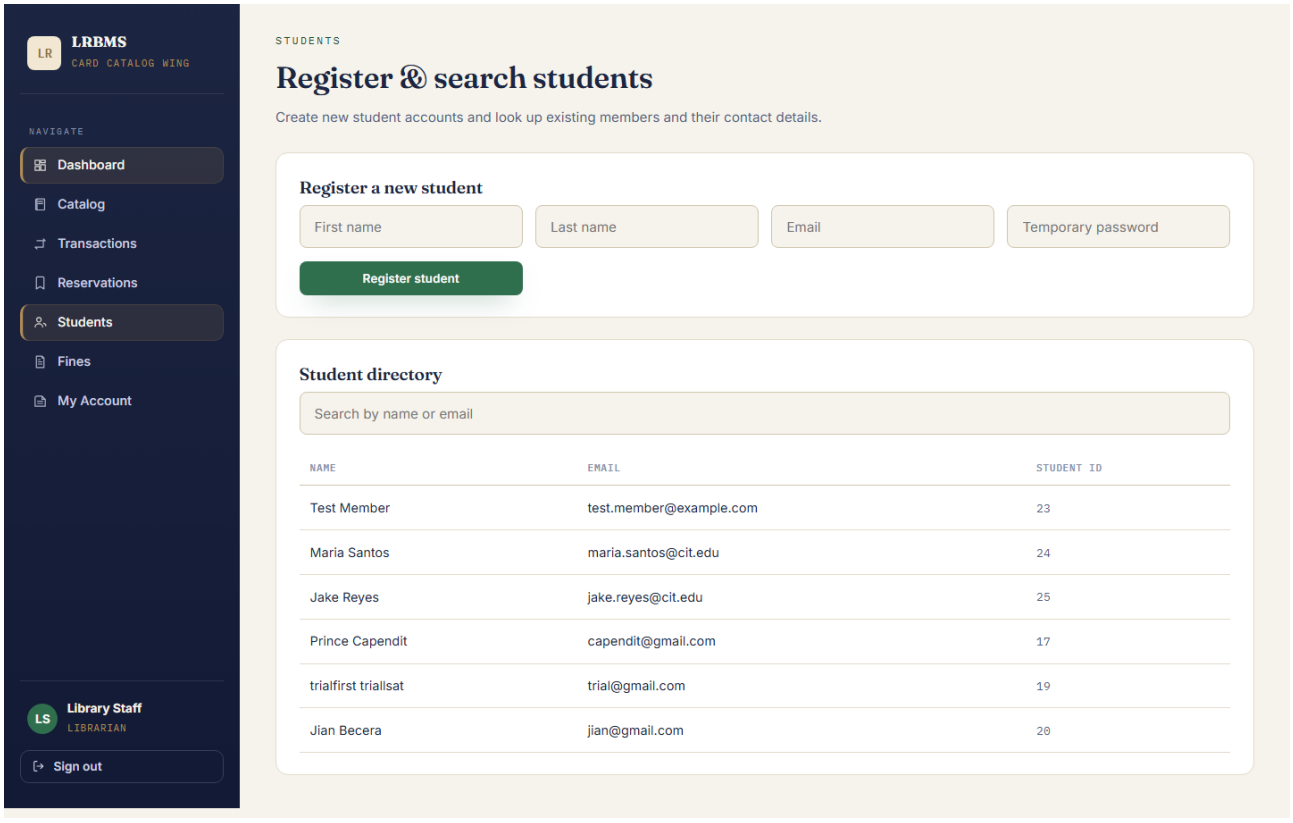
4.12 Check-In / Check-Out - librarian issues and returns loans (FR-007)



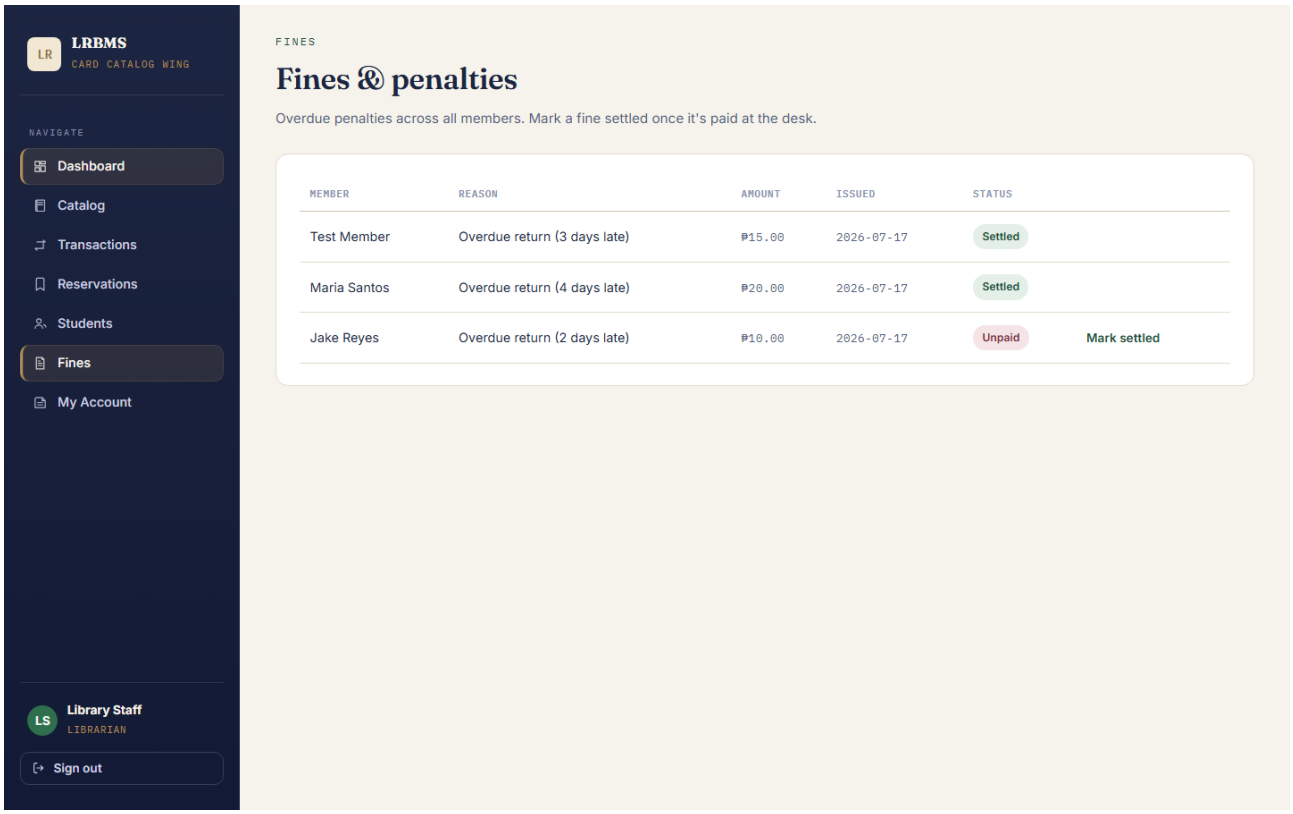
4.13 Reservation queue - approve/reject with duplicate prevention (FR-008)



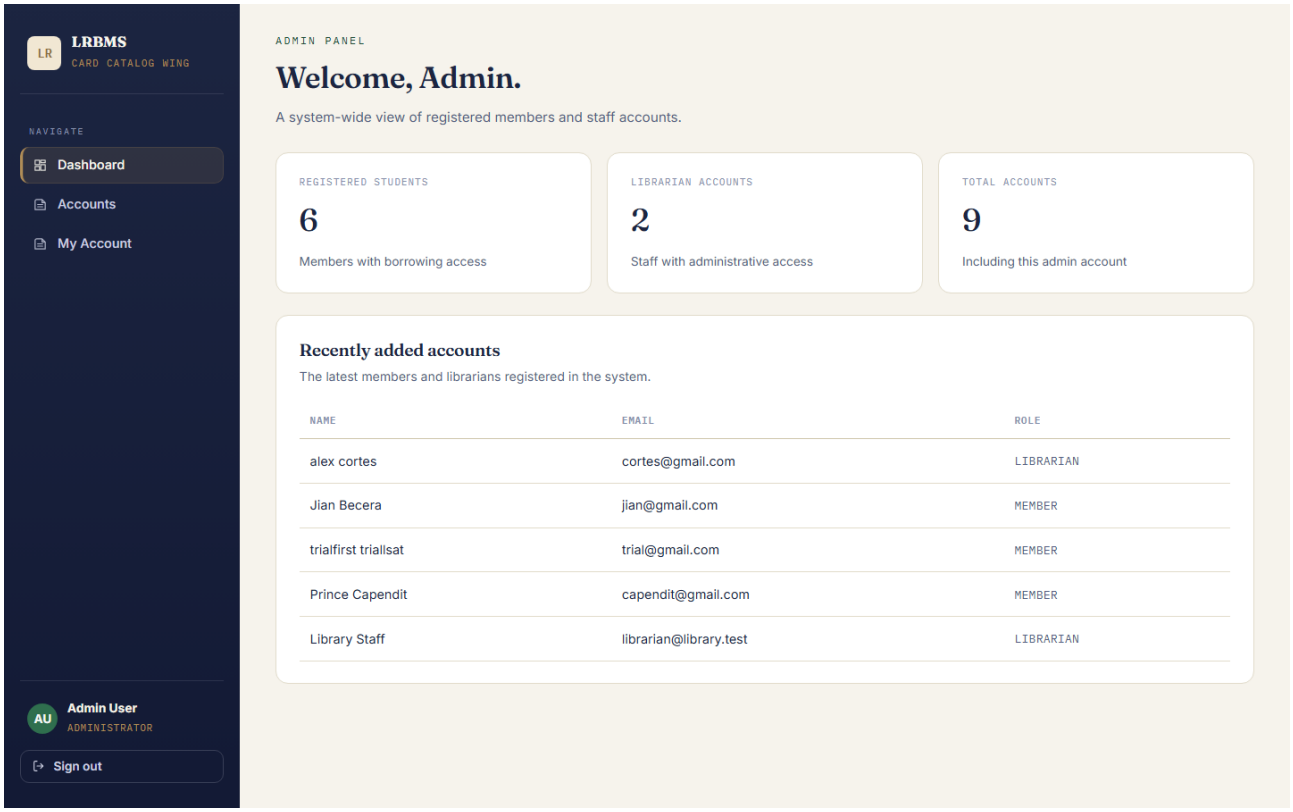
4.14 Students - librarian registers new members (FR-005)



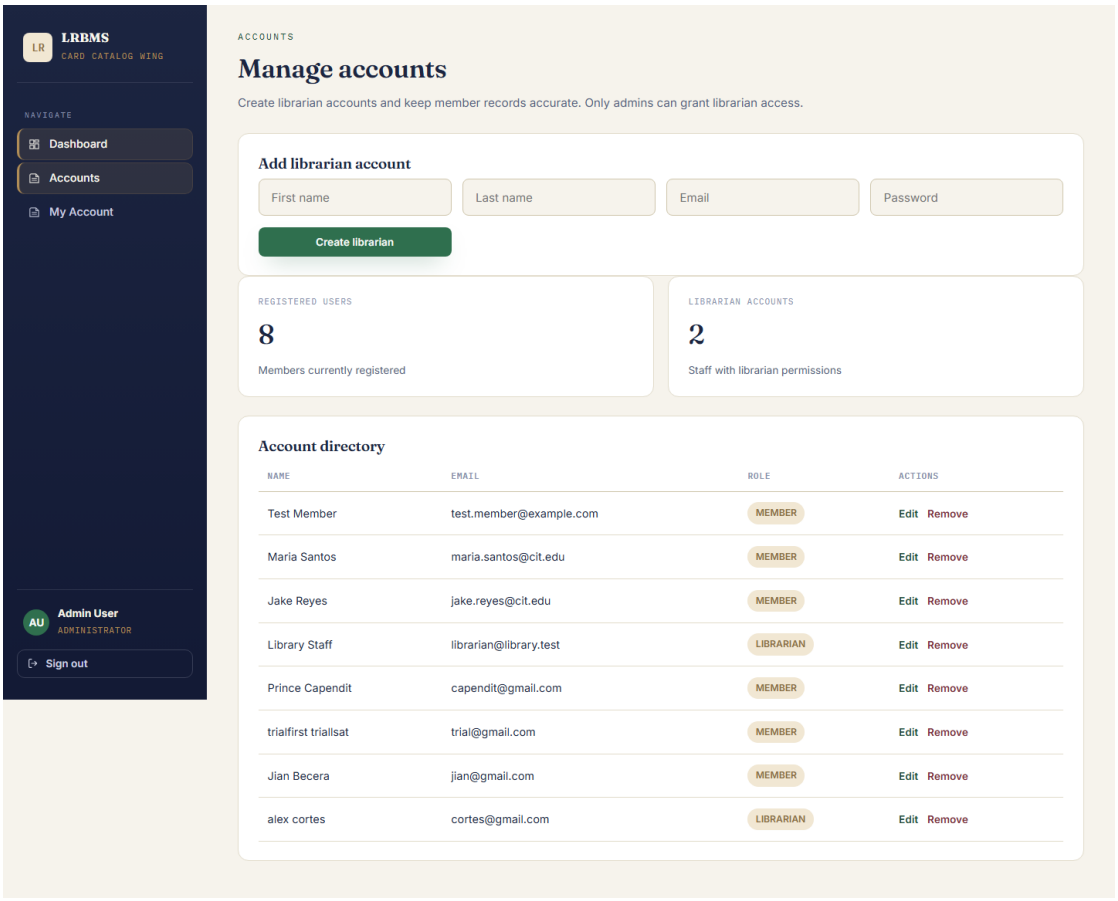
4.15 Fines & Penalties - settled and unpaid fines side by side



4.16 Admin Dashboard - system-wide registered accounts overview (FR-004)



4.17 Manage Accounts - admin creates librarian accounts, edits/removes members



5. End-to-End Workflow Test Results

Three automated, full-stack workflow tests run the real Spring Boot application against an isolated H2 database via @SpringBootTest + TestRestTemplate, exercising real HTTP requests through the actual controllers, services, and JPA repositories - not mocks. All three, plus the full 21-test backend suite, pass (mvn test, BUILD SUCCESS).

Workflow 1 - Registration through Check-In/Check-Out

`RegistrationToCheckoutWorkflowTest.studentRegistersLogsInAndLibrarianChecksOutAndInABook()`

A student self-registers (FR-005) and receives a generated Student ID; logs in (FR-001) and receives a JWT; the public catalog is searched with no authentication required (FR-002); a librarian logs in, adds a book (FR-006), and checks it out to the student (FR-007); the student reads their own loan back via an ownership-scoped endpoint; the librarian checks it back in and catalog availability is restored. Result: PASS - exercises FR-001, FR-002, FR-005, FR-006, FR-007 as one integrated path.

Workflow 2 - Reservation Waitlist (includes a validation scenario)

`ReservationWaitlistWorkflowTest.memberJoinsWaitlistAndDuplicateReservationIsRejected()`

A member joins the waitlist on a fully-checked-out title (succeeds, PENDING). VALIDATION SCENARIO - duplicate record prevention: the same member reserving the same title again is rejected with HTTP 409 and an explanatory error message. A further business-rule check rejects reserving a title that still has copies available. A librarian then approves one reservation and rejects a second member's reservation on the same title. Result: PASS - exercises FR-008 end to end including the required duplicate-prevention validation scenario.

Workflow 3 - Authorization and Access-Control Validation

`AuthorizationValidationWorkflowTest.roleEscalationIsBlockedAndStaffOnlyDataIsProtected()`

VALIDATION SCENARIOS in one integrated path: (a) invalid action - anonymous self-registration requesting role=ADMIN is silently downgraded to MEMBER server-side; (b) unauthenticated access - a request to a protected endpoint with no token is rejected; (c) unauthorized access - a logged-in MEMBER is rejected (403) from the staff-only member list, from creating a catalog resource, and from reading another member's profile, while reading their own profile succeeds; (d) integration failure handling / cross-feature business rule - an overdue checkout is checked in (generating a Fine), and the same member's subsequent reservation attempt is rejected specifically because of the unpaid fine, proving the Fine/Transaction/Reservation integration holds under a real HTTP call rather than a mocked unit test. Result: PASS - exercises FR-004 plus the required validation-scenario coverage.

Full backend suite: 21/21 tests passing (18 unit tests across Auth/Membership/Catalog/Transaction/Reservation/Fine services, plus the 3 workflow tests above). mvn test - BUILD SUCCESS.

6. Integration Issues Encountered and Solutions Applied

Both issues below were invisible to API-level testing (curl and TestRestTemplate, which don't trigger browser CORS preflight or realistic concurrent load) and were only found by driving the actual running application through a real browser for the screenshot walkthrough in Section 4 - which is itself the value of end-to-end, as opposed to purely unit-level, testing.

Issue 1: Browser login silently failed (CORS preflight rejected)

Symptom: logging in from the real React app in a browser failed with a CORS network error, even though the same request succeeded from curl or an automated test client. Root cause: the Spring Security rules were scoped by HTTP method (e.g. permitAll on POST /api/auth/login only). A browser's OPTIONS preflight request to that same path doesn't match a POST-scoped rule, so it fell through to the default authenticated() rule and was rejected before any CORS headers could be attached - the browser then reported the request as blocked by CORS policy. Solution: added an explicit CorsConfigurationSource wired into the security filter chain and permitted OPTIONS globally, since preflight requests never carry credentials and must be allowed regardless of the target endpoint's own rules. Verified with a manual OPTIONS request (Access-Control-Allow-Origin now present) and a full re-run of the browser walkthrough with zero console errors afterward.

Issue 2: Intermittent 403/500 errors under load (PgBouncer + JDBC prepared statements)

Symptom: identical, valid requests to the same endpoint intermittently failed - a stress test showed a public GET endpoint returning 200 most of the time but 403/500 unpredictably under repeated calls. Root cause: the datasource points at Supabase's PgBouncer connection pooler running in transaction pooling mode, which does not guarantee the same physical Postgres backend connection is used across statements on a pooled session. The PostgreSQL JDBC driver's default behavior names and caches server-side prepared statements per connection; when PgBouncer handed the pooled session to a different backend mid-session, the driver's cached prepared-statement names went stale or collided, surfacing as "prepared statement ... already exists" / "does not exist" SQL errors that aborted the transaction and bubbled up as request failures. Solution: added prepareThreshold=0 to the JDBC connection URL, which disables server-side named prepared statements entirely - the standard, documented fix for a JDBC client behind a transaction-mode PgBouncer pooler. Verified with a 25-request stress test against the same endpoint (25/25 succeeded) and a full re-run of the backend test suite (21/21 still passing).

7. Commit History

Each feature or integration task in this development cycle has its own commit with a descriptive message explaining the why, not just the what. Full history: https://github.com/JBecera/Becera_LMS_backend/commits/main

Feature / Integration Task	GitHub Commit
Backend cleanup - remove unused Resource/Booking scaffold	d9fe26b
Backend - JWT authentication & role-based authorization (FR-004)	c1f011a
Backend - Check-in/Check-out, Reservations, Fines + business rules (FR-003/007/008)	cbb5fb5
Frontend - app shell, JWT auth wiring, account management (FR-003)	1d6611a
Frontend - catalog, borrowing, reservations, fines UI (FR-002/006/007/008)	2f8a7ad
Mobile - session persistence & API layer	896a189
Mobile - member-facing screens (catalog/borrowing/reservations/fines/account)	219a608
Test - E2E workflow: registration through check-in/check-out	ad36ee2
Test - E2E workflow: reservation waitlist + duplicate-prevention validation	263efc5
Test - E2E workflow: authorization/access-control validation	f2a87c5
Fix - CORS preflight handling in Spring Security	ef8f93e
Fix - PgBouncer prepared-statement handling	af9ccd3

8. Individual Contribution Statement

This is an individually-developed capstone project. Jian Brenz C. Becera is the sole student developer responsible for requirements analysis, system design, and implementation across the backend (Spring Boot), web frontend (React), and mobile application (Kotlin/Jetpack Compose).

AI-assisted development disclosure: Claude Code (Anthropic) was used as a pair-programming assistant for this development cycle - implementing the backend domain logic, security layer, frontend pages, mobile screens, automated tests, and this report under the student's direction and review. Every change was reviewed, tested (21 automated backend tests plus a full manual/browser walkthrough of every screen), and is explainable: the business rules implemented (borrowing limits, fine calculation, reservation eligibility, role-based access control) map directly to the SRS's stated functional requirements and business rules, and the two integration bugs documented in Section 6 were diagnosed from first principles by reading the actual error output, not guessed at.

The student takes full ownership of and can explain the submitted source code, its architecture, and the reasoning behind each implementation decision described in this report.